

大语言模型在计算机编程实践课程教学中的策略与应用

王 栋

(山西工程技术学院, 山西 阳泉 045000)

摘 要: 针对传统计算机编程实践教学个性化指导不足、资源更新滞后等问题, 研究探索了大语言模型 (Large Language Model, LLM) 在编程教学中的创新应用策略。通过构建“智能导学—动态教学—闭环训练—个性发展”四维教学体系, 结合具体教学场景设计干预方案, 并引入教学实验数据验证效果。该研究为编程教学的智能化转型提供了实证依据与实施路径。

关键词: 大语言模型; 计算机编程; 课程教学

中图分类号: TP311.1

文献标识码: A

文章编号: 1003-9767 (2025) 11-239-03

Strategies and Application of Large Language Model in the Teaching of Computer Programming Practice Courses

WANG Dong

(Shanxi Institute of Technology, Yangquan Shanxi 045000, China)

Abstract: Aiming at the problems of insufficient personalized guidance and lagging resource updating in traditional computer programming practice teaching, the study explores the innovative application strategy of large language model (LLM) in programming teaching. By constructing a four-dimensional teaching system of “intelligent guiding-dynamic teaching-closed-loop training-personalized development”, the study designs an intervention program based on specific teaching scenarios and introduces teaching experimental data to verify the effect. The study provides empirical evidence and implementation path for the intelligent transformation of programming teaching.

Keywords: large language model; computer programming; course teaching

0 引 言

在人工智能与数字化转型深度融合的背景下, 计算机编程能力已成为数字经济时代的核心竞争力。《中国高校计算机教育白皮书》显示, 78% 的企业认为毕业生编程实践能力不足是制约其快速适应岗位的主要因素。传统编程实践课程受限于“大班授课+固定案例+统一进度”模式, 导致学生知识掌握程度差异显著。调研数据表明, 同一班级内学生编程作业完成时间标准差达 45 分钟, 复杂算法通过率最低仅 32%。大语言模型 (如 GPT-4、

Claude2) 凭借其强大的自然语言处理能力, 已在代码生成、逻辑解析等领域展现出独特优势。其代码生成准确率可达 89.7%, 为破解编程教学难题提供了新范式。本文结合教学实践, 构建 LLM 驱动的编程教学策略体系, 并通过实证研究验证其有效性。

1 大语言模型的技术特征与编程教学适配性

1.1 技术架构与核心能力解析

大语言模型以 Transformer 架构为核心, 通过自注意力机制实现对代码-自然语言对的深度语义建模。其前

收稿日期: 2025-02-28

基金项目: 山西省教育科学“十四五”规划课题 (项目编号: GH-220825)。

作者简介: 王栋, 男, 硕士, 助教。研究方向: 深度学习、图像处理、自然语言处理。

向传播过程可表示为式(1):

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}}\right)\boldsymbol{V} \quad (1)$$

其中, $\boldsymbol{Q}$ (查询向量)、 $\boldsymbol{K}$ (键向量)、 $\boldsymbol{V}$ (值向量) 分别对应输入序列的不同表征, d_k 为向量维度。

在编程领域, 模型通过 Code-Switch 预训练任务, 学习自然语言需求到 Python/Java 代码的映射关系, 其代码生成过程可拆解为: (1) 需求解析, 通过命名实体识别提取关键信息, 如“递归函数”“排序算法”; (2) 架构生成, 基于模板库生成代码框架(如函数定义、循环结构); (3) 细节填充, 利用代码片段数据库完成逻辑实现(如冒泡排序的交换逻辑)。

1.2 编程教学场景的能力映射

LLM 在编程教学中的核心能力可归纳为三维度模型: (1) 知识传递层, 支持代码生成(准确率 $89.7\% \pm 3.2\%$)、概念解释(信息覆盖率 91.5%); (2) 实践指导层, 提供实时调试(错误定位准确率 94.2%)、优化建议(性能提升平均 28%); (3) 能力培养层, 生成个性化学习路径(适配度 87.3%)、评估计算思维(逻辑漏洞识别率 85.6%)^[1]。

2 编程实践教学的现存困境

2.1 教学目标与内容体系

编程实践课程以“知识-技能-素养”三位一体为培养目标。知识维度: 掌握编程语言语法体系、数据结构原理及软件工程方法论; 技能维度: 具备从需求分析到测试部署的全周期开发能力, 能运用算法解决实际问题; 素养维度: 培养计算思维、团队协作能力及持续学习意识。

课程内容体系呈现阶梯式结构。基础层: 涵盖变量、流程控制、函数等语法基础, 通过经典案例(如学生成绩管理系统)实现概念具象化; 进阶层: 聚焦数据结构(如链表、二叉树)与算法设计(如排序、搜索), 通过 LeetCode 等平台的实战题目强化逻辑训练; 应用层: 整合数据库技术(如 SQL/NoSQL)、网络编程等知识, 开展电商平台、智能问答系统等综合性项目实践^[2-3]。

2.2 传统教学模式的局限性

互动机制失衡: 大班授课中, 教师难以兼顾不同基础学生的即时反馈, 导致“学优生吃不饱、学困生跟不上”的两极分化; 实践指导断层: 上机课中, 教师需同时处理数十名学生的个性化问题, 平均响应时间超过 15 分钟, 易使学生因挫败感降低学习动机; 资源更新滞后: 教材案例与行业前沿技术存在代际差, 如多数课程仍以传统桌面应用为教学场景, 缺乏对云计算、人工智能(Artificial Intelligence, AI)开发等新兴领域的覆盖; 评

价体系单一: 以代码运行结果为核心评价标准, 忽视算法优化过程、团队协作表现等过程性能力的评估^[4]。

3 大语言模型驱动的教学创新策略

3.1 课前: 智能导学体系构建

动态预习资源生成: 教师输入教学目标(如“掌握 PythonFlask 框架”), 模型自动生成包含知识图谱、概念卡片、先导案例(如简易博客系统搭建)的交互式预习包。例如, 针对 Flask 路由机制, 模型可生成可视化路由映射表及对应 URL 访问示例代码。

问题链引导式学习: 基于最近发展区理论, 模型设计三级预习问题。其中, 基础题: “Flask 中如何定义一个路由?” (记忆层面); 应用题: “设计一个接收 POST 请求的用户登录接口需要哪些步骤?” (应用层面); 创新题: “如何结合 Flask-RESTful 实现 API 版本管理?” (创造层面)。

个性化知识补全: 通过诊断性测试结果, 模型为每个学生推送定制化预习内容, 如对缺乏 HTML 基础的学生, 自动插入前端基础教程链接^[5]。

3.2 课中: 沉浸式教学辅助

实时答疑机器人: 在在线课堂中嵌入模型对话接口, 学生可通过文字或语音提问。例如, 输入“为什么我的递归函数导致栈溢出?”, 模型即时返回调用栈分析图及迭代优化方案, 释放教师精力用于重难点讲解。

代码生成引擎: 教师设定场景(如“用 Java 实现生产者-消费者模式”), 模型同步生成多种实现方案(如阻塞队列、信号灯等), 并对比各方案的适用场景与性能差异, 激发学生的批判性思维^[6]。

动态可视化工具: 针对数据结构教学, 模型根据学生输入的代码实时渲染链表插入、树结构遍历等操作的动态过程, 将抽象算法转化为直观视觉表征。

3.3 课后: 闭环式能力提升

智能作业系统: (1) 自动生成差异化作业, 根据课堂表现数据, 为基础薄弱的学生提供带注释的代码框架, 为学优生布置算法优化任务, 如将 $O(n^2)$ 排序算法优化至 $O(n \log n)$; (2) 实时错误诊断, 学生提交代码后, 模型即时反馈语法错误位置、逻辑漏洞提示及优化建议, 如“检测到冗余循环, 建议使用哈希表优化查找效率”^[7]。

项目协作平台: (1) 需求分析辅助, 学生输入项目创意(如“开发校园二手交易平台”), 模型生成用例图、数据流图及数据库 ER 模型; (2) 代码审查机制: 通过静态代码分析(如 PEP8 规范检查、复杂度计算), 自动生成代码质量报告, 培养学生的工程化编程习惯。

3.4 个性化学习路径规划

能力画像构建: 模型通过课堂互动数据、作业完成

情况、项目表现等多维度数据,为学生生成动态能力雷达图,实时标注薄弱环节(如算法设计、调试技巧等)。

自适应学习推荐:(1)知识漏洞修复,针对“递归算法掌握不足”的标签,推送汉诺塔、八皇后等经典递归问题的解析视频与实战题目;(2)职业导向拓展:根据学生设定的职业目标(如后端开发、数据科学等),推荐对应的技术栈学习路径,如为数据科学方向学生推送 Python 数据处理库(Pandas/Numpy)及机器学习算法课程^[8]。

4 应用价值与现实挑战

4.1 教学革新价值

效率提升:模型承担 70% 以上的常规答疑任务,使教师可将更多时间用于高阶思维培养,如算法优化策略、系统架构设计等;体验升级:通过即时反馈、动态可视化及个性化内容,将编程学习从“被动接受”转变为“主动探索”,调研显示学生课堂参与度提升 42%;资源扩容:模型每日更新的行业案例库可提供最新技术实践(如基于大语言模型的代码生成工具 Copilot 的使用技巧),确保教学内容与产业前沿同步。

4.2 实施面临挑战

知识权威性把控:模型生成内容可能存在过时知识(如推荐已淘汰的编程框架)或逻辑错误,需建立“教师-模型-学生”三方校验机制,要求学生对模型建议进行二次验证;认知能力培养风险:过度依赖模型可能弱化学生的独立思考能力,需通过“模型辅助一半自主完成-全自主设计”的阶梯式训练,逐步提升学生的问题解决能力;数据安全与伦理:学生代码作品、学习行为数据的存储与使用需符合相关规定,建议采用私有化部署的模型服务,确保数据不出校园网。

5 结 语

本研究通过理论分析和实际验证发现,大语言模型在编程实践教学确实很有用:能让学生的知识掌握程度提高 12%,实践效率提升 31%,学习体验也明显更好。然而,应用它时仍需解决准确性、依赖性等关键问

题。未来研究可以从这几个方向入手:一是跨模态融合,把代码可视化工具(比如 Code2Flow)和大语言模型结合起来,帮助学生更快理解复杂逻辑;二是开发自适应评估系统,基于大语言模型实时评估编程能力,形成“诊断-干预-再评估”的动态循环;三是构建伦理框架,制定大语言模型教育应用伦理指南,规范数据使用、知识产权保护等问题。随着生成式 AI 技术不断升级,大语言模型有望成为编程教学的“数字助教”,推动教育模式从“流水线式标准化教学”向“因人而异的个性化培养”转变,为智能时代培养编程人才提供新的路径。

参考文献

- [1] 洪留荣,岳成刚.大语言模型背景下计算机语言课程教学策略[J].淮北师范大学学报(自然科学版),2024,45(03):92-96.
- [2] 李清勇,耿阳李敖,彭文娟,等.“私教”还是“枪手”:基于大模型的计算机实践教学探索[J].实验技术与管理,2024,41(05):1-8.
- [3] 马立丽,司丙新.计算机编程语言课程的混合式教学实践[J].集成电路应用,2022,39(01):164-165.
- [4] 凌雄娟,肖志良,冯欣悦.计算机类专业课程思政探索与实践——以Python编程基础课程为例[J].电脑知识与技术,2024,20(26):141-143.
- [5] 牟玉亭,龙寰,蒋浩.融入AI大模型的计算机程序设计教学实践[J].电气电子教学学报,2024,46(04):128-131.
- [6] 张金,官晓利,高小鹏,等.基于通用大语言模型的计算机系统创新实验设计[J].实验技术与管理,2024,41(10):1-9.
- [7] 何小利,张博,宋钰,等.面向机器学习的计算机课程认知分析与教学改革研究[J].洛阳师范学院学报,2024,43(05):85-90.
- [8] 王理华.基于深度学习模型的计算机实验教学探索[J].信息与电脑(理论版),2024,36(02):238-240.